INtimeカーネルの管理するメモリ確保AllocateRtMemory APIがあり、対になるのはFreeRtMemoryだ。同様にMapRtSharedRtMemory/FreeRtMemoryの関係。これはRustでラッピングすることでメモリ開放、所有権管理を解決させることが可能か？

AllocateRtMemory / MapRtSharedMemory と FreeRtMemory は、Rustでラップして「所有権＋自動解放」に乗せることが十分可能です。ただし、そのラッパの内部は`unsafe`で書く必要があります。[1] [2] [3]

何ができるか（結論）

- `AllocateRtMemory`で確保した領域
  → Rust側で専用の RAII 型（例:`RtBox<T>` や `RtAlloc`）を定義し、`Drop`で `FreeRtMemory` を呼ぶことで、「スコープを抜けたら必ず解放」を実現できます。[2] [1]
- `MapRtSharedMemory`でマップした共有メモリ
  → 同様に `RtMappedRegion`のような型を作り、`Drop`で `FreeRtMemory` を呼んでマッピング解除できます（INtimeの仕様上、共有メモリ自体は解放されず、マッピングだけが消える）。[4] [3] [1]

具体的なラッパ設計イメージ

単純な確保ラッパ

```
#[repr(C)]
struct RtHandle; // 実際は不要・型のみ示すなら空

extern "C" {
    fn AllocateRtMemory(size: u32) -> *mut core::ffi::c_void; // 実際のシグネチャに合わせて調
    fn FreeRtMemory(addr: *mut core::ffi::c_void) -> u32;
}

pub struct RtAlloc {
    ptr: *mut u8,
    size: usize,
}

impl RtAlloc {
    pub fn new(size: usize) -> Option<Self> {
        unsafe {
            let p = AllocateRtMemory(size as u32) as *mut u8;
```

```
            if p.is_null() {
                None
            } else {
                Some(RtAlloc { ptr: p, size })
            }
        }
    }

    pub fn as_ptr(&self) -> *mut u8 {
        self.ptr
    }
}

impl Drop for RtAlloc {
    fn drop(&mut self) {
        unsafe {
            if !self.ptr.is_null() {
                let _ = FreeRtMemory(self.ptr as *mut core::ffi::c_void);
            }
        }
    }
}
```

- これで`RtAlloc`がスコープを抜けるとき、自動的に`FreeRtMemory`が呼ばれます。[1] [2]
- 「AllocateRtMemoryで取ってFreeRtMemoryで解放する」というINtimeのルールをRustの型が表現する形になります。[4] [1]

共有メモリマッピングのラッパ

```
extern "C" {
    fn MapRtSharedMemory(handle: u32, flags: u32, size: u32) -> *mut core::ffi::c_void;
    fn FreeRtMemory(addr: *mut core::ffi::c_void) -> u32;
}

pub struct RtMappedRegion {
    ptr: *mut u8,
    size: usize,
}

impl RtMappedRegion {
    pub fn map(handle: u32, size: usize) -> Option<Self> {
        unsafe {
            let p = MapRtSharedMemory(handle, 0, size as u32) as *mut u8;
            if p.is_null() {
                None
            } else {
                Some(RtMappedRegion { ptr: p, size })
            }
        }
    }

    pub fn as_slice(&self) -> &[u8] {
        unsafe { core::slice::from_raw_parts(self.ptr, self.size) }
    }
```

```
    pub fn as_mut_slice(&mut self) -> &mut [u8] {
        unsafe { core::slice::from_raw_parts_mut(self.ptr, self.size) }
    }
}

impl Drop for RtMappedRegion {
    fn drop(&mut self) {
        unsafe {
            if !self.ptr.is_null() {
                let _ = FreeRtMemory(self.ptr as *mut core::ffi::c_void); // マッピング解除
            }
        }
    }
}
```

- INtimeのドキュメント通り、`MapRtSharedMemory`で作られたマッピングは`FreeRtMemory`で解除しますが、物理メモリ自体は解放されません。[3] [1] [4]
- Rust側からは「`RtMappedRegion`が生きている間だけ共有メモリにアクセスできる」という所有権モデルに置き換わります。

どこまで所有権を Rust に任せられるか

- Rustが確保したものは Rust が解放
  - `AllocateRtMemory`を呼ぶのを Rust に一本化すれば、そのメモリの寿命は`RtAlloc`型が完全に管理できます。C側には`RtAlloc`の中身（生ポインタ）だけを渡し、「解放は Rust 側 API を呼べ」とルールを決めれば、ダブルフリーやリークを防げます。[5] [4]
- 共有メモリ（MapRtSharedMemory）の場合
  - 共有メモリそのものは「CreateRtMemoryHandle / CloseRtMemoryHandle」など別のライフサイクルを持つため、Rust側の`RtMappedRegion`は「マッピングの寿命だけ管理」する扱いになります。[3] [4]
  - これはUnixの`mmap/munmap`をラップする Rust ライブラリと同じパターンで、よく使われている手法です。[6]
- クロス言語所有権
  - Rust側でRAIIを効かせるためには、「Cが勝手に FreeRtMemory を呼ばない」「ポインタのコピー・解放権限をどちらに持たせるか」をAPI設計時に明確化する必要があります。[7] [5]
  - 一般的には「Cからも使える`rte_free_*`のようなC関数をRust側で提供し、Cが解放したくなったらそれを呼ぶ」形にすると所有権の流れがはっきりします。[8] [7]

まとめると

- Yes:`AllocateRtMemory / MapRtSharedMemory / FreeRtMemory`はRustでラッピングして、所有権と自動解放（Drop）に乗せることが可能です。[1] [4] [3]
- ただしそのラッパ実装は`unsafe`を含み、
  - オフセットとサイズの整合性
  - INtimeの「ブロック全体をFreeする」というルール

    - 共有メモリの場合は「物理メモリは解放されない」をきちんと守る必要があります。[5] [4] [1]
- 一度`RtAlloc`や`RtMappedRegion`のような型にしてしまえば、その上にRT-Edgeのタグ管理・デスクリプタ管理を安全なRustコードとして積み上げることができ、Cからは従来通りポインタベースのAPIで共有する、という構成が取れます。[6] [8] [5]

もし INtime SDK の`AllocateRtMemory`/`MapRtSharedMemory`の正確なシグネチャを貼ってもらえれば、それに合わせた完全な`extern "C"`宣言と、RT-Edge向けのラッパ型の雛形をRustコードで書き起こします。

⁂

1. https://support.tenasys.com/7-0/freertmemory
2. https://support.tenasys.com/7-2/allocatertmemory
3. https://support.tenasys.com/6-2/maprtsharedmemory
4. https://support.tenasys.com/7-0/ovw_memory
5. https://www.reddit.com/r/rust/comments/r25q6p/ffi_and_custom_nonglobal_allocator/
6. https://www.reddit.com/r/rust/comments/1pqm8al/safety_of_shared_memory_ipc_with_mmap/
7. https://users.rust-lang.org/t/c-ffi-memory-leak-take-ownership-of-allocated-memory-in-c-c/24337
8. https://oneuptime.com/blog/post/2026-01-26-rust-ffi-c-interoperability/view
9. https://www.mxt.jp/PDF/INtime5catalog.pdf
10. https://gitioc.upc.edu/platform/soem/-/blob/9b562d9f53af6582e302c63f0014fbfc4445ed8f/oshw/intime/nicdrv.c
11. https://www.mnc.co.jp/INtime_doc/10_23.htm